```
import random

def merge_sort( zz):
     '''
     Sorts list zz in ascending order and returns
     the sorted list.
     Implementation by R.E.Hilburger (c) 2025
     '''
     iiHalf = len(zz) // 2
     zzA = zz[ : iiHalf]
     zzB = zz[ iiHalf :]
     if iiHalf > 30:     # 30 is somewhat arbitrary
          zzA = merge_sort( zzA)  # recursion
          zzB = merge_sort( zzB)  # recursion
      else:
          selection_sort( zzA)
          selection_sort( zzB)
          #end

      # merge zzA and zzB to zz:
      #
     zz = []
     while True:
          if len( zzA) == 0 or len( zzB) == 0:
               break
           # Here, both lists are non-empty:
          if zzA[ 0] > zzB[ 0]:
               # append zzB, it is smaller:
               zz.append( zzB.pop(0))
           else:
               # append zzA, it is smaller
               # or the same size:
               zz.append( zzA.pop(0))
               #end
          continue
     if len( zzA) == 0 and len( zzB) == 0:
          # if both equal zero, then zz is complete
          return zz
      # so, exactly one is > 0
     if len( zzA) > 0:
          return zz + zzA   # concatenate
     return zz + zzB   # concatenate

def selection_sort( zReal):
     '''
     Sort list zReal in ascending order.  This function
     can sort integers, floats, and strings.  `ii` ranges
     through a region of list zReal looking for the
     largest element.  When the pass ends at `iiBorder`,
     a conditional swap is made.  The largest element is
     deposited at the current border.  Elements are sorted
     above the border, and unsorted below the border.
     This process iterates over the unsorted part, which
     gets smaller and smaller.  `selection_sort` is faster
     than `bubble_sort` due to fewer swaps.
     '''
     iiBorder = len( zReal)
     swap_count = 0
     while True:
          vBig = zReal[ 0]
          iiBig = 0
          for ii in range( 1, iiBorder):  # while ii ranges
               if zReal[ ii] > vBig:    # get "biggest"
                    vBig = zReal[ ii]
                    iiBig = ii
                    #end
               continue
          if ii != iiBig:
               # swap:
               swap_count += 1
               zReal[ iiBig], zReal[ ii] = zReal[ ii], vBig
               #end
          iiBorder -= 1
          if iiBorder < 0:
               break
          continue
     print( " selection_sort swap_count:", swap_count)
     return

def get_unsorted_list():
     random.seed( "Prevue is remarkable")
     zz = []
     cnt = 1500
     while cnt > 0:
          cnt -= 1
          # inclusive of both args:
          val = random.randint( -2000, 2000)
          zz.append( val)
          continue
     return zz

def bubble_sort( zz, pA, pC):
     '''
     Sort list zz only from indices pA to pC inclusive.
     Sort in place and ascending.  Can sort integers,
     floats, and strings.
     '''
     assert pA <= pC     # proceed if this is true

     if pA == pC:   # one item is already sorted
          return     # it takes 2 items to be unsorted!

     swap_count = 0
     for pX in range( pA, pC):
          for pY in range( pX+1, pC+1):
               if zz[ pY] < zz[ pX]:
                    # swap:
                    swap_count += 1
                    zz[ pX], zz[ pY] = zz[ pY], zz[ pX]
                    #end
               continue
          pA += 1
          if pA == pC:
               print( " bubble_sort swap_count:", swap_count)
               return
          continue
     return

zz = get_unsorted_list()
# print( " The unsorted list zz:", zz)
print( " len( zz):", len( zz))
zz1 = list( zz)     # copy zz to zz1
zz2 = list( zz)     # copy zz to zz2
zz3 = list( zz)     # copy zz to zz3
zz4 = list( zz)     # copy zz to zz4

zz4 = sorted( zz4)  # Python's built-in sort

zz1 = merge_sort( zz1)
if zz1 == zz4:
     print( " merge_sort worked")
     #end

bubble_sort( zz2, 0, len(zz2)-1)  # sort all of zz2
if zz2 == zz4:
     print( " bubble_sort worked")
     #end

selection_sort( zz3)
if zz3 == zz4:
     print( " selection_sort worked")
     #end

```